
Experimental study of preference modeling techniques.

Internship Report

Tutor: Mr. Olivier Strauss

Written by: Curtis Martelet

2025-08-25

Sommaire

I.	Introduction	4
1.	Presentation of the internship environment	4
2.	Scientific objective of the project	4
II.	Tools and fundamental concepts	5
1.	Additive measure	5
1.1.	Definition	5
2.	Non-additive measure	5
2.1.	Definition & Motivation	5
2.2.	The Game	5
2.3.	Monotonicity	6
2.4.	Special case: <u>The Capacity</u>	6
3.	Choquet Integral	6
3.1.	Definition	6
3.2.	Special Case	7
3.3.	The Möbius Transform	7
III.	Objective and scope of the project	8
1.	Scientific objective	8
1.1.	Comparing different learning methods	8
1.2.	Evaluating accuracy and behavior/evolution under different parameters	8
2.	Application of the project	9
2.1.	Practical interest (ranking, scoring of objects, robustness, etc.)	9
2.2.	Identified limitations	9
IV.	Implemented calculations	10
1.	Tested learning models	10
1.1.	<u>Linear</u>	10
1.2.	<u>Game</u>	10
1.3.	<u>2-Additive Game</u>	11
1.4.	<u>MacSum</u>	11
1.5.	<u>Centered MacSum</u>	11
1.6.	<u>Capacity (ADMM-RDA)</u>	12
2.	Loss functions	12
2.1.	Quadratic Loss (L2)	13
2.2.	Absolute Loss (L1)	13
2.3.	ADMM-RDA	14
3.	Scores	14
3.1.	Predictions	14
3.2.	Score Calculation	14
V.	Experiments & Results	16
1.	Experimental Protocol	16
2.	Results Obtained and Comparative Analysis	16
2.1.	Comparison for different Δ of Game, MacSum, Centered MacSum	17
2.2.	L1 Function	17
2.3.	ADMM-RDA	19

2.4. Additional Observations	19
VI. Conclusion & Perspectives	20
1. Project Conclusion	20
2. Personal Conclusion	20
3. Possible improvements and perspectives for future work	20
VII. References	21

I. Introduction

1. Presentation of the internship environment

The LIRMM

The **LIRMM** (Laboratory of Computer Science, Robotics, and Microelectronics of Montpellier) is a joint research unit of the **University of Montpellier** and the **CNRS**. It brings together more than 400 people (including professors, researchers, PhD students, and postdoctoral researchers) and covers three major scientific fields: computer science, robotics, and microelectronics.

The laboratory develops research ranging from fundamental concepts to industrial and medical applications, in collaboration with many academic and socio-economic partners, both in France and internationally.

The ICAR team

As part of my internship, I joined the **ICAR** (Image & Interaction) team of the LIRMM.

This team brings together researchers in robotics and computer science around topics related to imaging and, more broadly, visual data. It currently has nine permanent researchers (University of Montpellier and CNRS), joined by regular collaborators from the Montpellier University Hospital (medical imaging), the TETIS laboratory (remote sensing), and the CIRAD (agronomic modeling).

The team is led by Prof. **Olivier Strauss**, professor at the university, assisted by Dr. **Noura Faraj**, associate professor at the University of Montpellier.

The work is structured into four main areas:

- **Analysis & Processing**
- **Multimedia Security**
- **Modeling & Visualization**
- **Artificial Intelligence for visual data**

This research covers topics such as medical imaging, image security and authentication, statistical processing of visual data, as well as deep learning applied to image analysis and modeling.

My supervisor during this internship was Prof. **Olivier Strauss**. This internship is part of the **Modeling & Visualization** area.

2. Scientific objective of the project

The project falls within the field of multicriteria decision-making.

During this internship, I studied different non-additive measures, and I was tasked with making comparisons between them.

The objective is to design and test an online algorithm capable of learning to provide preferences from pairs of data supplied in a sequential and random manner.

II. Tools and fundamental concepts

Before presenting the different learning methods tested, it is important to define the fundamental notions I worked with during this internship.

1. Additive measure

1.1. Definition

An additive measure is a function that assigns a weight to each subset of a set of criteria. This measure satisfies the following property:

$$\mu(A \cup B) = \mu(A) + \mu(B) \text{ si } A \subseteq B$$

Probabilities are a special case of additive measure where the sum of the weights of all subsets is equal to 1. Thus, the following property holds:

$$\mu(\Omega) = 1$$

2. Non-additive measure

2.1. Definition & Motivation

Formally, a measure μ on a set Ω must satisfy $\mu(\emptyset) = 0$.

It does not impose the additive property, meaning that for two disjoint subsets A and B of Ω ($A \cap B = \emptyset$), it is not necessarily true that $\mu(A \cup B) = \mu(A) + \mu(B)$.

Unlike additive measures, non-additive measures make it possible to model situations where criteria interact with each other (complementarity or redundancy).

For example, two criteria A and B may, taken individually, have moderate importance ($\mu(A)$ and $\mu(B)$ small), but together form a synergy where $\mu(A + B)$ is much greater than the sum of their individual importances.

Conversely, if two criteria overlap in their influence, the capacity may assign to their coalition¹ a value lower than the sum, reflecting redundancy. Thus, the capacity “represents importance weights for each group of criteria” rather than only for each criterion taken independently.

2.2. The Game

In non-additive measure theory, the term **Game** is often used to refer to a function that assigns a value to each subset of criteria.

A game is a function $\mu : 2^\Omega \rightarrow [0, 1]$ which, as explained in the previous point, associates a weight with each subset of criteria, without necessarily respecting the property of additivity.

¹a **coalition** refers to any subset of criteria considered together, i.e., a group of criteria whose interaction or combination is taken into account by the non-additive measure

The only condition for a game is that the empty set has a zero value: $\mu(\emptyset) = 0$.

2.3. Monotonicity

Monotonicity is a fundamental property of non-additive measures used in multicriteria decision-making.

Intuitively, a measure is monotone if adding criteria (i.e., considering a larger set of satisfied criteria) cannot reduce the overall evaluation. Formally, if A and B are two sets of criteria such that $A \subseteq B$, then the measure μ satisfies $\mu(A) \leq \mu(B)$.

In other words, if an alternative fulfills at least all the same requirements as A and in addition satisfies other criteria (set B), its “score” evaluated by μ will not be lower.

Simply put, **considering (or satisfying) more criteria must not lead to a lower score.**

A simple example would be the evaluation of several mobile phones according to criteria such as camera quality, battery life, and price.

A first phone that satisfies the price criterion obtains a certain score $\mu(\text{price})$; if another phone satisfies both price and camera quality, it should obtain a score $\mu(\text{price, camera})$ at least as high.

Monotonicity therefore prevents counterintuitive situations where the addition of an advantage (an extra satisfied criterion) would lower the overall evaluation.

2.4. Special case: The Capacity

The capacity is a special case of non-additive measure.

It is a function that satisfies the following properties:

- Empty set $\mu(\emptyset) = 0$
- The full set of criteria $\mu(\Omega) = 1$
- Monotonicity respected, such that $\mu(A) \leq \mu(B)$ whenever $A \subseteq B$
- $\mu(A) \geq 0 \forall A \subseteq \Omega$

Thus, a capacity is a game that respects monotonicity, non-additivity, and guarantees that the set of all criteria will always have the maximum weight.

3. Choquet Integral

3.1. Definition

The Choquet integral is the aggregation operator defined for use when dealing with a Game. It allows combining the values of the criteria while taking into account the possible interactions between them. Unlike a weighted sum, the Choquet integral considers not only the individual importance of each criterion but also that of coalitions of criteria.

Formally, for an evaluation vector $x = (x_1, \dots, x_n)$ sorted in increasing order ($x_{(1)} \leq \dots \leq x_{(n)}$), the Choquet integral with respect to a capacity μ is defined as:

$$C_{\mu(x)} = \sum_{i=1}^n (x_{(i)} - x_{(i-1)}) * \mu(\{(i), \dots, (n)\})$$

with $x_{(0)} = 0$, $\{(i), \dots, (n)\}$ being the coalition of the $n - i + 1$ criteria with the best evaluations.

3.2. Special Case

If the Game is additive (i.e., a classical measure), the Choquet integral reduces to a weighted average: $C_{\mu(x)} = \sum_{i=1}^n x_i * \mu(i)$

3.3. The Möbius Transform

The Möbius transform provides an alternative representation of a capacity. Rather than assigning a weight directly to each coalition via μ , it defines Möbius coefficients m such that:

$$\mu(A) = \sum_{B \subseteq A} m(B)$$

Link between the Choquet integral and the Möbius transform

Using the Möbius transform, the Choquet integral can be written as a linear sum:

$$C_{\mu(x)} = \sum_{A \subseteq \Omega} m(A) * \min_{i \in A} x_i$$

This formulation is important because it facilitates the implementation of algorithmic processes such as optimization, regularization, and learning.

III. Objective and scope of the project

1. Scientific objective

This internship is part of a research project initiated by Prof. Strauss, Dr. Rico, and Dr. Marsala.

Its goal is to develop a learning method capable of determining, without bias, whether there is a preference or an equivalence between objects according to several criteria.

As part of my internship, I worked on the testing and evaluation of the **MacSum** model. A secondary objective was also to study the risks of overfitting associated with this new method.

1.1. Comparing different learning methods

The main objective of the project is to **compare different learning methods** for multicriteria decision-making. More specifically, the algorithm under study must, based on several criteria, be able to compare two objects A and B , and determine either a **preference** relation ($A > B$), or an **indifference** relation ($A \approx B$).

The internship consisted of confronting several approaches:

- **Linear method:** simple and fast, but limited to situations where the relationship between criteria and preference is linear.
- **Game method:** allows modeling all interactions between criteria. However, its main drawback is complexity: the complete game requires 2^n parameters for n criteria, which quickly becomes intractable; even restricted versions (e.g., 2-additive) remain costly.
- **MacSum method** (*developed within the team*): more parsimonious, it only requires n parameters for n criteria.

The goal is to determine whether this method can reach a level of accuracy comparable to the **Game** method.

- **Centered MacSum method** (*developed within the team*): a variant of MacSum that maintains the same expected accuracy but with one fewer parameter (the threshold is no longer required), which simplifies the learning process.

With this method, the goal is to see whether it can reach the same accuracy as the **MacSum** method.

- **Capacitive method** (*developed by another team*): it uses a capacity to model interactions between criteria.

We aim to compare this method with the **MacSum** method to see if they achieve similar performance.

1.2. Evaluating accuracy and behavior/evolution under different parameters

The scientific evaluation therefore focuses both on the accuracy of the methods and on their behavior under different parameters (number of criteria, size of the preference base, normative constraints, etc.).

We will observe the average error over the iterations, as well as Recall, Precision, and Accuracy.

2. Application of the project

2.1. Practical interest (ranking, scoring of objects, robustness, etc.)

From a practical perspective, this type of algorithm has many applications across various domains.

To cite several examples, it could:

- Be used in the medical field,
where it would allow determining whether one medical treatment is preferable to another according to, for example, the patient's current health condition, ongoing treatment, age, etc.
- Be used to rank products,
based on objective criteria such as price, origin, chemical composition, etc.
- Be used in geology,
to classify soils according to criteria such as water content, mineral composition, porosity, etc.

2.2. Identified limitations

However, this type of algorithm presents certain limitations.

The first concerns the nature of the input data: the criteria must necessarily be numerical. For instance, it is not possible to directly provide letters or symbolic categories.

IV. Implemented calculations

In this fourth part of the report, we present the different learning models tested. Each model is associated with a loss function that guides its learning by measuring the gap between the model's predictions and the training data.

Thus, the evaluation relies on a systematic pairing: each learning method is paired with one of the two loss functions.

1. Tested learning models

1.1. Linear

The linear model is the simplest reference among the approaches tested.

Its principle is straightforward: it consists in using the criteria as they are, without transformation or enrichment of the variable space. Concretely, the input criteria vector is simply copied and transmitted, which amounts to considering each criterion independently, with its own weight.

Size of the parameter vector:

- Size of the parameter vector: $p = n$
- Parameter vector: $\varphi \in \mathbb{R}^p$
- Function $\varphi(x)$ identity, $X = \varphi(x) = x$

Thus, the output is given by a simple dot product:

$$y = \langle \varphi, x \rangle$$

1.2. Game

The Game model considers all possible coalitions of criteria.

- Size of the parameter vector: $p = 2^n - 1$
- Parameter vector: $\varphi \in \mathbb{R}^p$
- Function $\varphi(x)$: for each subset $S \subseteq \Omega$

$$\Phi_S(x) = \min_{i \in S} (x_i)$$

Thus, we obtain a vector $\Phi(x)$ of size $2^n - 1$.

The set of coalitions is represented by a binary matrix B of size $(2^n - 1) \times n$, where each row corresponds to a coalition and each column to a criterion.

For example, for $n = 4$:

$$B = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

1.3. 2-Additive Game

A variant, called 2-Additive Game, restricts the construction to coalitions of size 1 and 2 (single criteria + pairs).

- Size of the parameter vector: $p = n + \binom{n}{2} - 1$
- Parameter vector: $\varphi \in \mathbb{R}^p$
- Function $\varphi(x)$: for each subset $S \subseteq \Omega$ with $|S| \leq 2$

This method significantly reduces the complexity of the Game while keeping a large number of interactions between the criteria.

Thus, if we take the example with $n = 4$, we obtain:

$$B = \begin{pmatrix} 0 & 0 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

1.4. MacSum

The MacSum model is a nonlinear approach based on ranking the criteria and constructing a transformed vector $X = \varphi(x)$. Unlike the linear model, each input variable is not used as is: its contribution depends on its relative position with respect to the other criteria (minimum, maximum, etc.).

- Size of the parameter vector: $p = n$
- Parameter vector: $\varphi \in \mathbb{R}^p$
- Function $\varphi(x)$: specific transformation given by the construction algorithm described below, $X = \varphi(x)$

The MacSum algorithm is provided in the appendix.

1.5. Centered MacSum

One of the drawbacks of the previous models lies in their sensitivity to learning parameters (learning rate, score tolerance Δ , data tolerance δ , etc.). Centered MacSum aims to mitigate this dependence by removing a key parameter, Δ .

Its principle is no longer to produce a single value as a score, but **an interval** representing the uncertainty zone around the prediction.

For comparison, the algorithm then relies on their respective intervals, according to three cases:

- **Inclusion**: one interval is entirely contained within the other.
- **Overlap**: the two intervals partially overlap.
- **Dominance**: one interval is strictly larger than the other.

It is the choice of ordering that determines whether a situation corresponds to preference or indifference:

- In **weak order**:
 - **Inclusion**: Indifference
 - **Overlap**: Preference. The upper bounds of the intervals x and x' are compared to decide whether $x > x'$ or $x < x'$.
- In **strong order**:
 - **Overlap**: Indifference
 - **Dominance**: Preference. As in weak order, the upper bound is examined to determine whether the preference is inverted.

With the **Imprecise MacSum** method, we use only one loss function: L1 (which will be detailed later).

- Size of the parameter vector: $p = n$
- Parameter vector: $\varphi \in \mathbb{R}^p$
- Function $\varphi(x)$: specific transformation given by the construction algorithm.

1.6. Capacity (ADMM-RDA)

The Capacity model seeks to directly learn a capacity, i.e., a monotone and normalized non-additive measure. It combines two complementary methods:

- RDA (Regularized Dual Averaging): updates the parameters using the average of accumulated sub-gradients, ensuring good stability in online learning.
- ADMM (Alternating Direction Method of Multipliers): projects the parameters after each RDA update to guarantee that the capacity constraints (monotonicity and normalization) are always respected.
- Size of the parameter vector: $p = 2^n - 1$
- Parameter vector: $\varphi \in \mathbb{R}^p$
- Function $\varphi(x)$: for each subset $S \subseteq \Omega$, the sum of the negatively transformed criteria is computed.

Before training, a data normalization step is applied to improve convergence: each variable is transformed into three components (low, medium, high), obtained by linear interpolation between the quartiles of the distribution. This transformation bounds all variables in $[0, 1]$.

2. Loss functions

In the context of preference learning, we have a set of pairs of alternatives (x, x') with an associated preference relation: $x > x'$ (preference) or $x \approx x'$ (indifference).

We denote:

$$y = \langle \varphi, \Phi(x) \rangle; y' = \langle \varphi, \Phi(x') \rangle$$

where $\Phi(x)$ is the transformation of the criteria according to the chosen model (linear, Game, MacSum, etc.), and φ the weight vector.

Learning consists of adjusting φ to minimize a loss function $L(x, x', \varphi)$ that measures the discrepancy between the prediction (y vs y') and the observed preference ($x > x'$ or $x \approx x'$).

2.1. Quadratic Loss (L2)

The quadratic loss, called **L2**, is based on the sum of squared errors. For a pair (x, x') :

$$L_2(x, x', \varphi) = \begin{cases} (\delta - (y - y'))^2 & \text{if } x > x' \\ (\llbracket y - y' \rrbracket - \delta)_+^2 & \text{otherwise} \end{cases}$$

The update of φ is performed by gradient descent:

$$\varphi_{L_2} = \begin{cases} \varphi - \beta \cdot (\Phi(x') - \Phi(x)) \cdot [\delta - \Delta]_+ & \text{if } x > x' \\ \varphi - \beta \cdot \text{sign}(\Delta) \cdot (\Phi(x) - \Phi(x')) \cdot \llbracket \Delta \rrbracket - \delta]_+ & \text{otherwise} \end{cases}$$

with $\Delta = y - y'$ and β the learning rate.

2.2. Absolute Loss (L1)

The absolute loss, called **L1**, uses the sum of absolute errors.

It is defined as:

$$L_1(x, x', \varphi) = \begin{cases} [\delta - y + y']_+ & \text{if } x > x' \\ \llbracket |y - y'| - \delta \rrbracket_+ & \text{otherwise} \end{cases}$$

The parameter update is based first on the computation of Γ :

$$\Gamma = \begin{cases} \begin{cases} \text{If } x > x' \\ (\Phi(x') - \Phi(x)) & \text{if } \Delta < \delta \\ 0 & \text{otherwise} \end{cases} \\ \begin{cases} \text{If } x \approx x' \\ 0 & \text{if } |\Delta| < \delta \\ \text{sign}(\Delta) \cdot (\Phi(x) - \Phi(x')) & \text{otherwise} \end{cases} \end{cases}$$

with $\Delta = y - y'$, and 0 a zero vector of the same size as φ .

Then, an online average is computed:

$$\bar{\Gamma} = \bar{\Gamma} + \frac{1}{t} \cdot (\Gamma - \bar{\Gamma})$$

where t is the current iteration.

The final update is:

$$\varphi \leftarrow -\text{sign}(\bar{\Gamma}) \cdot \frac{\sqrt{t}}{\gamma} \cdot [\|\bar{\Gamma}\| - \beta]_+$$

with β the learning rate, γ a regularization hyperparameter, and t the current iteration.

2.3. ADMM-RDA

The ADMM-RDA method uses the L1 loss, but with a more sophisticated update that combines:

- RDA (Regularized Dual Averaging): an online learning method equivalent to the Follow-the-Regularized-Leader (FTRL) approach. Unlike classical gradient descent methods, RDA updates the parameters based on the average of accumulated sub-gradients.
- ADMM (Alternating Direction Method of Multipliers): an iterative optimization method, suitable for batch problems. It decomposes the optimization into several simpler subproblems, solved at each iteration using “decoupling” variables.

The ADMM-RDA algorithm is provided in the Reference.

3. Scores

3.1. Predictions

Each element of the dataset is associated with a **reference score**, which is used to generate the training examples:

- If the difference between two scores is **greater** than a threshold Δ , we consider that there is a **preference**.
- If the difference is **less than or equal** to Δ , we consider that there is an **indifference**.

The algorithm learns to reproduce these relations from the input criteria.

During evaluation, the same principle is applied to the **scores predicted by the model**:

1. the predicted scores are calculated for the two compared elements,
2. they are compared using the tolerance δ ,
3. it is verified whether the obtained relation (preference or indifference) matches the one derived from the reference scores.

This mechanism makes it possible to determine whether the algorithm has correctly reproduced the preferences observed in the reference data.

In my program, I chose to explicitly handle only the case where x is preferred to x' . This simplifies the score calculation by reducing the number of cases to manage. If during random sampling $x < x'$, it is enough to invert the two rows to reduce the case to $x > x'$.

3.2. Score Calculation

To evaluate performance, several classical classification indicators were used:

- **Precision**: proportion of correct predictions among all positive predictions made by the model.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

If Precision = 1, all predictions are correct.

- **Recall**: proportion of positive cases correctly identified among all positive cases.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

If Recall = 1, the model does not miss any positive case.

- **Accuracy**: proportion of correct predictions (positive and negative) over the entire dataset. It is a global performance measure.

$$\text{Accuracy} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$$

In the case of our project, we have 5 possible outcomes:

- True Indifference: The model correctly predicts that $x \approx x'$
- False Indifference: The model predicts $x \approx x'$ when it should have been $x > x'$
- True Preference: The model correctly predicts that $x > x'$
- False Preference: The model predicts $x > x'$ when it should have been $x \approx x'$
- Inverse Preference: The model predicts $x' > x$ when the true preference is $x > x'$

V. Experiments & Results

1. Experimental Protocol

The experiments were carried out using the public housing price dataset from Boston <https://www.cs.toronto.edu/~dave/data/boston/bostonDetail.html>.

The protocol followed is as follows:

- The dataset is randomly split into two parts:
 - **70%** of the data for training,
 - **30%** for model evaluation.
- For each model, **5000 independent simulations** are performed.
 - Each simulation includes **1000 training iterations** followed by **1000 testing iterations**.
 - The learning rate is fixed at 10^{-5} .
 - The parameter Γ is set to $1e^3$.
 - Seven values of Δ are studied: 10^{-1} , 10^{-2} , 10^{-5} , 10^{-8} , 10^{-10} , 10^{-12} , 10^{-15} .
 - The ρ parameter of ADMM-RDA is set to 0.3.

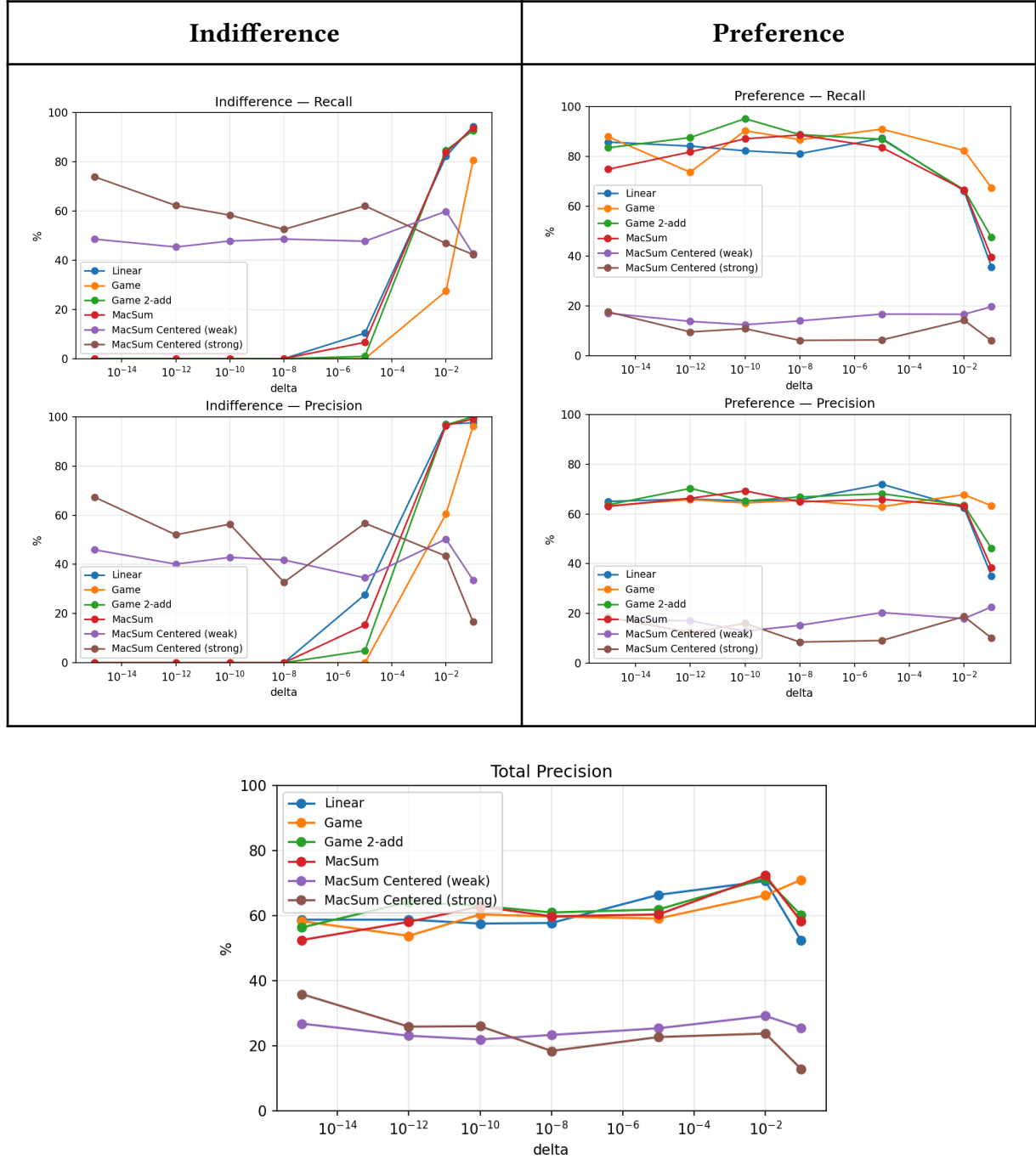
2. Results Obtained and Comparative Analysis

In this section, I will present the results obtained with the different models and loss functions.

We will examine the outcomes obtained with the test data.

2.1. Comparison for different Δ of Game, MacSum, Centered MacSum

2.2. L1 Function



Observation:

From these plots, we can observe the variation of Recalls and Precisions of the different learning methods as a function of Δ .

The first observation that can be made is the variation of scores between preferences and indifferences. When Δ values are small, the preference scores of all models are above 80%, indicating mostly correct predictions. On the other hand, the Indifference scores are extremely poor (0%).

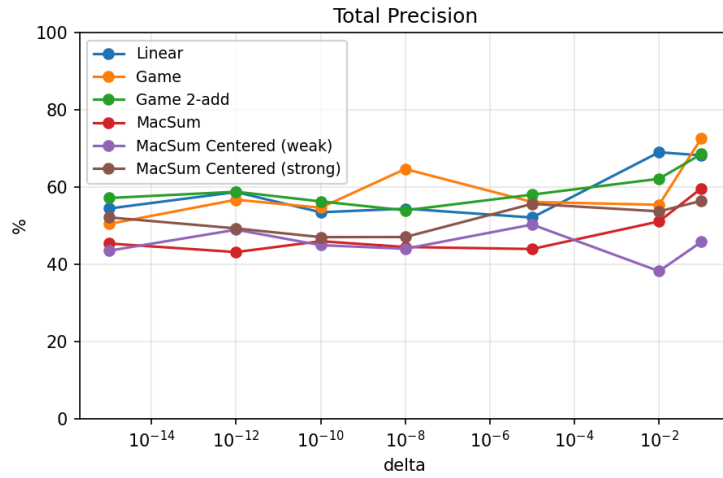
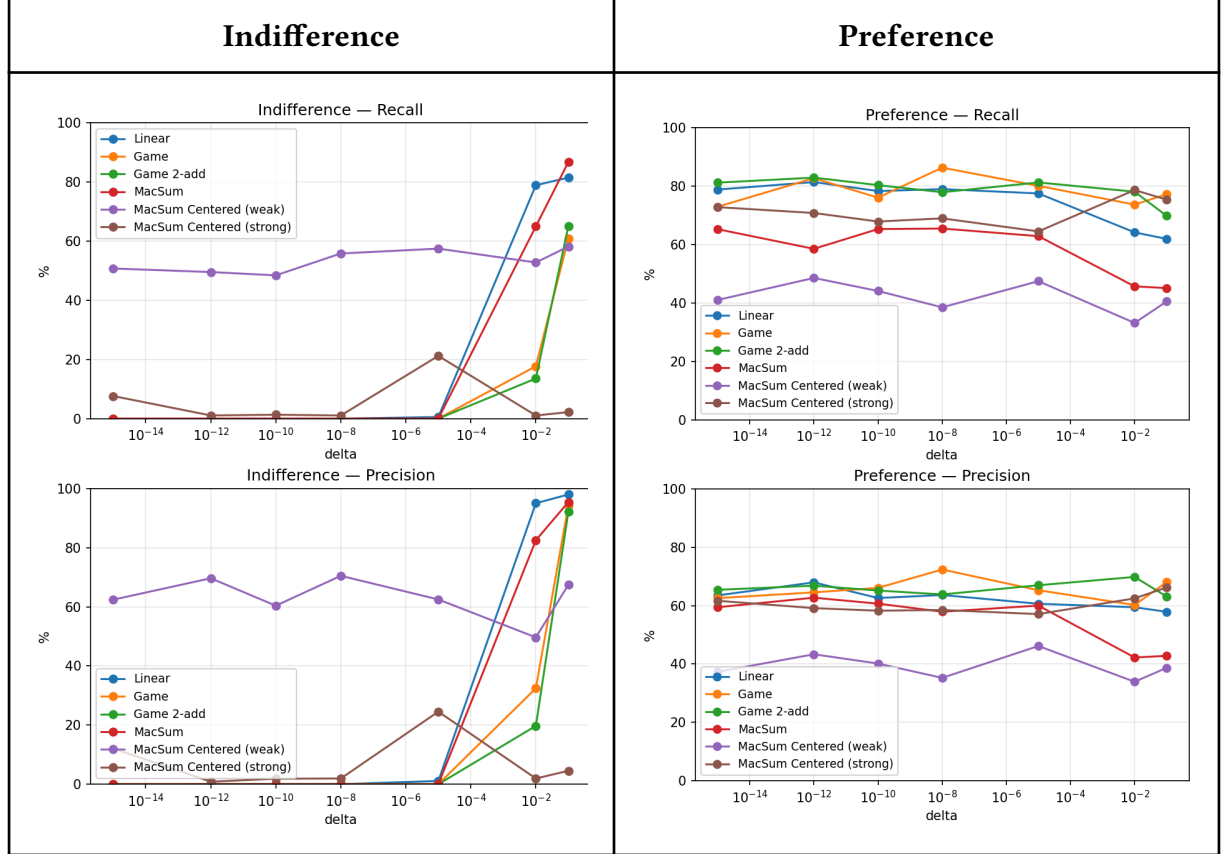
We can therefore conclude that the algorithm classifies all comparisons as Preferences.

When Δ increases ($1e-2$ and $1e-1$), the scores are inverted: all Indifference scores are close to 100%, while Preference scores collapse.

Thus, the models predict that all comparisons are indifferences.

Finally, the **Centered MacSum** methods remain approximately constant: this observation is explained by the absence of Δ in the algorithm.

L2 Function



Observation:

Compared to the L1 function, the L2 function is much more stable. If the scores of the models in the case of indifferences are also around 0% when Δ is small, they reach 100%

when Δ is large.

Unlike the L1 function, the Preference scores do not collapse: they remain stable, between 80% for the best models (**Game** and **Centered MacSum (Strong)**) and 50% for the less effective ones (**MacSum** and **Centered MacSum (Weak)**).

As with the L1 function, the **Centered MacSum** methods remain approximately constant.

2.3. ADMM-RDA

Compared to the previous model, the ADMM-RDA model is much more sensitive to divergences. Thus, regardless of the Δ selected, I was not able to make the error of my model converge, which was not the case with the other learning models. However, I do observe convergence with a linear model (this model is used to test the program: if there is no convergence, then there is an issue with our code).

That said, it is highly likely that my program is incorrect: when I examine it in detail, I can spot NaN values or extremely high, mixed with more logical ones.

In its current state, I therefore cannot compare this learning method with the others.

2.4. Additional Observations

The number of iterations plays a crucial role in the convergence of the MacSum and Centered MacSum models:

- with too few training iterations, the model diverges more easily,
- with a sufficient number of training iterations but too few total iterations, the model converges but produces a majority of incorrect predictions.

During experiments on another dataset different from housing prices, a limitation was identified:

when the reference scores are too concentrated around a few close values (in the dataset, out of 1000 entries, the population only had 4, 5, and 6), the algorithm struggles to properly distinguish preferences. Too small differences then lead to more learning errors.

VI. Conclusion & Perspectives

1. Project Conclusion

In conclusion, we can observe that the MacSum learning model achieves results similar to other models: Game, 2-Additive Game, Linear.

Moreover, although the results are slightly lower than the others, the Centered MacSum methods provide acceptable outcomes.

2. Personal Conclusion

This internship allowed me to become familiar with the field of algorithmics through a concrete research project, carried out within the *ICAR* (Image & Interaction) team of the LIRMM. Being integrated into this team gave me the opportunity to discover how a research laboratory operates on a daily basis, both in terms of scientific organization and collaborative work.

I was able to discuss with students also doing their internship, but specializing in diverse areas such as computer science and imaging.

Working under the supervision of Prof. **Olivier Strauss**, I learned how algorithmic models are developed and tested.

Although I was frustrated by my program not working, this project introduced me to the study of multicriteria decision-making, a domain at the crossroads of mathematics, computer science, and artificial intelligence.

This immersion helped me to understand the importance of rigor in the design and evaluation of models, as well as the value of collaboration between experts from different backgrounds. It also strengthened my ability to approach a scientific problem methodically, to analyze results critically, and to place them in the broader context of ongoing research in the laboratory.

Finally, this internship also enabled me to develop my skills in C.

3. Possible improvements and perspectives for future work

A first step would be to correct the ADMM-RDA program in order to compare it with the other methods.

Secondly, it would be necessary to find more than one dataset compatible with these methods. The dataset I used was the only one (out of about ten) compatible with this type of learning, due to the nature of the constraints listed earlier.

VII. References

- O. Strauss, A. Rico, C. Marsala, C. Martelet - Comparing four different preference modeling
- Yassine Hmidy, Agnès Rico, and Olivier Strauss - Macsum aggregation learning
- Margot Herin, Patrice Perny, Nataliya Sokolovska - Online Learning of Capacity-Based Preference Models

Algorithm 1: MacSum (construction of X)

```
1: procedure MACSUM( $\phi$ ,  $x$ )
2:    $\triangleright$   $X$  : output, size  $n$ , initialized to 0
3:    $X \leftarrow \text{zero}(n)$ 
4:    $\triangleright$  1) Sort the indices according to  $\Phi$  (ascending)
5:    $\text{idx} \leftarrow \text{Sort } \Phi \text{ in ascending order}$ 
6:
7:    $\triangleright$  2) Forward pass: process  $\Phi \leq 0$ 
8:    $x_{\min} \leftarrow x[\text{idx}[1]]$ 
9:    $x_{\max} \leftarrow x[\text{idx}[1]]$ 
10:  if  $\Phi[\text{idx}[1]] \leq 0$  then
11:     $X[\text{idx}[1]] \leftarrow X[\text{idx}[1]] + x_{\min} + x_{\max}$ 
12:  end
13:  for  $t = 2 \rightarrow n$  do
14:    if  $\Phi[\text{idx}[t]] \leq 0$  then
15:       $X[\text{idx}[t]] \leftarrow X[\text{idx}[t]] - x_{\min} - x_{\max}$ 
16:    end
17:     $x_{\min} \leftarrow \min(x_{\min}, x[\text{idx}[t]])$ 
18:     $x_{\max} \leftarrow \max(x_{\max}, x[\text{idx}[t]])$ 
19:    if  $\Phi[\text{idx}[t]] \leq 0$  then
20:       $X[\text{idx}[t]] \leftarrow X[\text{idx}[t]] + x_{\min} + x_{\max}$ 
21:    end
22:  end
23:
24:   $\triangleright$  3) Backward pass: process  $\Phi \geq 0$ 
25:   $\text{idx} \leftarrow \text{reverse the order of idx}$ 
26:   $x_{\min} \leftarrow x[\text{idx}[1]]$ 
27:   $x_{\max} \leftarrow x[\text{idx}[1]]$ 
28:  if  $\Phi[\text{idx}[1]] \geq 0$  then
29:     $X[\text{idx}[1]] \leftarrow X[\text{idx}[1]] + x_{\max} + x_{\min}$ 
30:  end
31:  for  $t = 2..n$  do
```

```

32:   if  $\Phi[\text{idx}[t]] \geq 0$  then
33:   |    $X[\text{idx}[t]] \leftarrow X[\text{idx}[t]] - x_{\max} - x_{\min}$ 
34:   end
35:    $x_{\min} \leftarrow \min(x_{\min}, x[\text{idx}[t]])$ 
36:    $x_{\max} \leftarrow \max(x_{\max}, x[\text{idx}[t]])$ 
37:   if  $\Phi[\text{idx}[t]] \geq 0$  then
38:   |    $X[\text{idx}[t]] \leftarrow X[\text{idx}[t]] + x_{\max} + x_{\min}$ 
39:   end
40: end
41: return  $X$ 
42: end

```

Algorithm 2: ADMM-RDA

```

1: procedure ADMM-RDA( $\gamma, \lambda, \rho, T$ )
2:    $t \leftarrow 1$ 
3:    $\varphi_1, \mu_1, z_1 \leftarrow (0, \dots, 0)$ 
4:
5:   while  $t < T$  do
6:   |   ▷ receive pairwise example  $(x_t, y_t)$ 
7:   |   ▷ compute loss gradient  $g_t \in \partial \ell_t(m_t)$ 
8:   |   ▷ update average gradient  $\bar{g}_t$ 
9:   |    $g_t^\mu \leftarrow g_t - B^T(\bar{\mu}_t - \rho(B\bar{\varphi}_t - \bar{z}_t))$ 
10:  |    $\varphi_{t+1} \leftarrow -\frac{\sqrt{t}}{\gamma} [|\bar{g}_t| - \lambda]_+ \text{sign}(\bar{g}_t)$ 
11:  |    $z_{t+1} \leftarrow -\left[\frac{\mu_t}{\rho} - Bm_{t+1}\right]_+$ 
12:  |    $\mu_{t+1} \leftarrow \mu_t - \rho(Bm_{t+1} - z_{t+1})$ 
13:  end
14:  return  $\varphi_T$ 
15: end

```

With the following parameters:

- λ is a hyperparameter that controls the degree of regularization.
- γ is a hyperparameter, but its role is not clearly identified here (likely linked to the scaling of updates in relation to iteration t).
- ρ is a hyperparameter that controls the level of penalization.
- T is the number of iterations.

The matrix B is built in the same way as for generating a line of criteria with the *GAME* method, but the cells are filled with -1 .